IMPLEMENTATION PLAN · REV 2026-08-07

Implementation Plan — building the ~1.0T two-stage corpus

Written 2026-08-07. Companion to `docs/FINAL-DATASET-REPORT.md`, which decides *what* the corpus contains. This document decides *how it gets built*, and it is written to be read before any job is submitted.

Every claim here is graded. `MEASURED-IN-CODE` means a file and line in this repository says it. `MEASURED` means a real run or a real byte read produced the number. `DERIVED` means arithmetic from a measured input, with the inputs named. `CARD` means an upstream dataset card asserts it and nobody has checked. `UNVERIFIED` means plausible and unchecked — flagged, never used to justify a decision.

---

0. The executive summary, and the five things that would have gone wrong

The pipeline that will build this corpus already exists and already ran: 27 bundles, 10,049 shards, **251.2B tokens** published on 2026-08-05. Scaling it to 1.0T is mostly arithmetic — but five defects surfaced in this review, and **not one of them fails loudly.** Each either discards work silently or produces a corpus that passes every gate while being wrong.

#	blocker	consequence if unfixed	fix
1	**Ordinals shift when a source is added**	adding one 4B source renames **98% of shards** and voids **882B tokens**	freeze the full plan first — **0 code**
2	**The FinePhrase de-dup predicate is never called**	59.8B declared synthetic is **~18.5B distinct**; the epoch guard reports green at ~4× true exposure	~5 lines, at the reader
3	**The dedup set OOMs at 1T**	**27.92 GB for the DCLM bundle** in a 15.03 GB container (§5.2a)	flat `np.uint64` → 2.60 GB
4	**The decontamination index is built from 5-shot renders**	149,777 exact hashes are **dead** for MMLU/ARC/HellaSwag	rebuild from raw fields, one CPU job
5	**43% of all bytes moved is the val split**, serving 0.39% of the tokens	2.02× over-read; intrinsic to a per-document hash carve, not a formula bug	file-shard val bundles, ~30 lines

Two more that are not blockers but would embarrass us: `tokenizers` **is not a declared dependency**, so production resolves whatever PyPI served that morning — and every Cosmopedia document begins with a leading space, which changes its first token under byte-level BPE.

And the wall-clock (§8A), which is the same build either way:

	as the pipeline is configured today	with the fixes	floor
end to end	**~36 h**, and two stages fail their timeouts outright	**~10 h**	6.6 h of tokenize at the 128 vCPU cap

The 3.7× gap is **entirely flags and two constants** — no new architecture. Five threading facilities exist in the package and work; **every default is 1**, and the registered job definitions do not pass the flags. `verify --deep` is the clean example: it ran 1.005 TB in **3.27 h on one stream of a 16 vCPU box**, when the same code at `--hash-workers 8` was **measured at 7.82×** — about 25 minutes.

The two most instructive blockers are below; the rest are in their own sections.

The first is an ordering trap. The plan of record says to "ingest source by source, each as a separately-approved platform job." Executed literally, that discards nearly everything each time. A shard's ordinal comes from a single per-split counter walking the plan in alphabetical order (`corpus.py:352–359`, MEASURED-IN-CODE), so inserting a source shifts every source that sorts after it. Simulated on the real stage-1 mix: **adding one 4B-token source renames 35,278 of 35,998 shards — 98% — and voids 882B tokens**, about 23 hours of tokenize. The shard path carries the ordinal and the path is inside `manifest_sha256`, so a rename is a different dataset identity, and `bundle_is_done` rejects every prior receipt.

The fix costs no code: compute the complete plan for every source of both stages **before the first job**, then run bundles in whatever order approvals arrive. `--shard/--of` already slices bundles and `bundle_is_done` already skips finished ones. The consequence is real though — **the mix has to be final before the first token is written.** Freezing the mix, not starting the ingest, is the blocking step.

The second is a correctness gap with a green light on top of it. The four FinePhrase configs are one corpus rephrased four ways over the same ~339M FineWeb-Edu documents, **MEASURED at 91.0–92.9% pairwise id overlap**. The code to de-duplicate them exists, is tested, and is verified balanced to within 0.27 percentage points — and **production never calls it.** `keeps_id`'s three call sites are a reporting function, a test, and a measurement script. `IdSet.contains`, the anti-join primitive, has zero callers anywhere in the repository.

So of 59.8B declared synthetic tokens, roughly **18.5B are distinct source documents** (DERIVED from the measured overlap) and ~41B are rephrasings of documents already present under a different format label. No deduplication method catches this, at any scope, because four rephrasings of one document are four different strings. And the epoch guard reports deep green while it happens: `epochs_for` divides by the *declared* pool size, so a mixture drawing 0.25 from each of four synthetic sources reads 0.33–0.50 epochs on the dashboard while true per-document exposure is about **4×**.

⚠️ **Two things to hold straight here, because four different percentages circulate for this and they are not four findings.**

First, the distinct fraction is ONE measurement expressed two ways. All of these describe the same corpus:

figure	what it is	grade
91.0–92.9%	**pairwise id overlap between configs — the INPUT, not a distinct fraction**	MEASURED
26.83% distinct	complete-column read of 287,000 ids	**MEASURED — cite this one**
~28%	the same 26.83%, rounded, in `FINAL-DATASET-REPORT.md` §13	same measurement
30.9% (18.5 ÷ 59.8)	the same finding derived from token mass instead of id counts	DERIVED

They agree: four configs over one parent set would give exactly **25.0%** if the overlap were total, and 26.83% is a little above that. **26.83% and 30.9% are the same result by two methods, bracketing ~1/4 to ~1/3.** Nothing here disagrees with anything else.

Second — and this is the one that misleads — 59.8B is the RESERVOIR's synthetic pool, not this corpus's draw. `FINAL-DATASET-REPORT.md` §3 already applies the fix and takes **36.0B from a single FinePhrase partition**, which is why its table reads "*FinePhrase, one partition.*"

	tokens	distinct	why the difference
the reservoir, as built	59.8B across **4 configs**	~18.5B	the defect: four rephrasings of one parent set
this corpus, as planned	**36.0B from 1 partition**	**~36.0B**	one config, so nothing to overlap with

So blocker 2 is not "the corpus is 4× over-exposed" — it is "the code cannot yet express the plan the report already specifies." The report's 36B/one-partition design is correct on paper; `keeps_id` is what makes it true in the bytes, and it has **zero production callers**. If the partition is not wired in and someone draws 36B from all four configs, the exposure defect returns at ~2.4× on ~15B distinct documents.

A passing verification artifact for a code path production does not execute is the most dangerous shape a gap can take, because every audit looks green.

Both fixes must land before the plan is frozen, and the FinePhrase one must land before anything is tokenized: the registry's own trap text says that after tokenization there is no document→id mapping and it cannot be retrofitted.

---

1. What already works, so it does not get rebuilt

Worth stating plainly, because the temptation when a review finds two blockers is to distrust everything around them.

property	evidence
The plan is a pure function, content-addressed	`plan_document` takes no clock, no S3, no environment; `plan_id` is the sha256 of its own JSON. Two runs that agree on the plan agree on every key they will write. MEASURED-IN-CODE, `corpus_build.py:182–289`
The build is deterministic	Nine bundles re-run under `--force` reproduced **byte-identical digests**. MEASURED, `artifacts/reservoir/realized-tokens.json`
Resume is safe and honest	`bundle_is_done` re-HEADs every shard and compares **sizes**, not mere presence. MEASURED-IN-CODE, `corpus_build.py:387–427`
The val split cannot leak	`is_held_out` is a pure hash of `(source, doc_id)` with no PRNG, so both splits are decided from one read. MEASURED-IN-CODE, `corpus.py:368–402`
Memory is bounded during pack	The sink uploads each shard and drops it, so at most one ~100 MB buffer is resident. MEASURED-IN-CODE, `corpus_build.py:461–468`
The double-encode trap is already closed	`min_tokens` filters from the ids `encode_batch` already produced. The alternative cost 91% of build compute on 1 of 32 cores. MEASURED, `corpus_pack.py:230–250`
Ordinal reuse is not silently accepted	`allocate_ordinals` raises when a plan names one stream twice. MEASURED-IN-CODE, `corpus.py:340–348`

The determinism result is what makes the coarse resume unit survivable: a lost bundle can be re-run and will produce the same bytes.

---

2. The ordering constraints — why the steps cannot be reordered

Several of these are one-way doors. The build is a straight line, and each step below names what breaks if it moves.

#	step	why it is pinned here
0	**Stage sources to S3**	Optional but recommended (§3). Everything downstream reads in-region afterwards
1	**Freeze the complete plan**	Adding a source later renames 98% of shards (§0)
2	**Apply the FinePhrase id partition**	After tokenization there is **no document→id mapping**; it cannot be retrofitted
3	**Read + carve val**	`is_held_out` is per-document, so a val bundle must read the train documents beside it
4	**Dedup, then decontaminate**	Dedup is one sha256; decontamination is up to `len(words)-12` blake2b hashes. Cheap predicate first means a duplicate is never contamination-checked
5	**Neutralize boundary markers**	Must precede the encode. Afterwards the marker is an id indistinguishable from the boundary we append
6	**Tokenize, appending EOS**	The `min_tokens` floor applies here, from ids already computed — never a second encode
7	**Pack + upload**	One shard resident at a time
8	**Receipt**	Per bundle. This is the resume unit
9	**Publish to landing**	⚠️ A `manifest.json` fires EventBridge → **automatic promotion**. There is no publish-without-promote mode
10	**Gate A validation**	Recomputes from bytes and rejects on mismatch

Step 4's ordering is not a style preference: contamination checking is the expensive half, and duplicates are common in web crawls, so testing the cheap predicate first is a real saving.

Step 9 deserves emphasis because it is the one irreversible step. Writing a manifest **is** publishing. To stage without promoting, the validator job must be cancelled or the EventBridge rule disabled first.

---

3. Proposed change: stage the sources to S3 once

The problem. `_reader_for` streams live from HuggingFace inside the Batch container, and its own docstring flags this as the least-tested path in the build: "⚠️ *UNVERIFIED against live HF from inside a Batch container — every offline test injects* `documents=` *instead.*" MEASURED-IN-CODE, `corpus_build.py:900-906`.

The proposal. A pass 0 that copies raw source files from HuggingFace to `s3://edullm-landing/_src/<source>/`. No parsing, no filtering, no tokenizing. Every later pass reads in-region.

3.1 The read volume — and a claim of mine that a later audit correctly demolished

⚠️ **RETRACTED: my earlier finding that the pipeline "reads 18 TB to fetch 4.21 TB" was wrong**, and so was the fix I proposed for it. Recorded here rather than deleted, because the reasoning error is instructive and because task #25 was created on the strength of it.

What I claimed. `_reader_for` sets `budget = int(bundle.tokens * _CHARS_PER_TOKEN * _FILTER_HEADROOM / keep_rate)` with `_CHARS_PER_TOKEN = 6.0` and `_FILTER_HEADROOM = 1.5` (`corpus_build.py:865,881`) — i.e. 9.0 chars/token against a measured 4.31 B/token — and that a val bundle's `keep_rate = 0.005` divisor made the val split budget another 9 TB, for 18 TB total.

Why it is wrong, verified two ways.

1. `val_fraction` **cancels out of the read, algebraically.** A val bundle's tokens are `want × VF` and its divisor is `VF`; a train bundle's are `want × (1−VF)` over `(1−VF)`. **Both budgets equal** `want × 9.0` **exactly, for any** `val_fraction`**.** So the "200× divisor" is not an amplifier at all — I read a formula and failed to cancel it. Both source documents state the formula; neither cancels it.
2. **The budget is a CEILING that is never reached.** The reader is a lazy generator feeding `pack`, which iterates `stream_refs` and stops as soon as the planned shards are full (`corpus_pack.py:727–741`, `exhausted` flag). **Confirmed on the real run: 26 of 27 bundles filled every shard**, with unfilled refs only in `finewiki--train` (33 refs, 90.5% of plan) — so `pack` stopped before the reader exhausted its budget in every other case.

Consequence: correcting `_CHARS_PER_TOKEN` **would save exactly zero bytes** and carries pure downside — set it too low and a bundle starves, producing the unfilled refs that `verify` then rejects. **Do not make that change.**

What the real number is. The measured over-read is **2.02×**, and the genuinely surprising part survives: **43% of all bytes moved is the val split, serving 0.39% of the tokens.** That is not the budget formula's fault — it is intrinsic. `is_held_out` is a hash of the document id, so a val document cannot be reached without reading the train documents interleaved with it. The fix is file-sharding, not a constant.

The lesson, which is the same one §8A.1 draws about the calibration file: I derived a headline number from a formula without checking whether the code ever reaches it. A budget is not a measurement.

3.2 Then stage the sources once

	figure	grade
stage once	**4.21 TB** of real source text (already compressed as parquet / jsonl.gz)	DERIVED from 19 measured tok/byte values
S3 Standard storage	**~$97/month**, so ~**$194** for a two-month build window	DERIVED at $0.023/GB-month
HF → AWS transfer	**free** (AWS does not charge ingress)	—
re-read ~8.5 TB (4.21 TB × 2.02), **staged in-region**	**0.09–1.9 h** (0.24 h at 8 × 10 Gbit/s)	DERIVED
re-read ~8.5 TB, **live from HF**	**3.8–19 h** at 5–1 Gbit/s, and rate-limit exposed	DERIVED

⚠️ **And the HF figure may be an order of magnitude optimistic.** Both rows above assume bandwidth borrowed from an *S3* measurement. Reconciling the reservoir's measured 8 h build instead implies **HF CDN throughput near 8.4 MB/s**, which would make a live re-read of 8.5 TB take **~11 days**. That single unmeasured number is the strongest argument for staging, and Phase 0b measures it first.

Staging is what makes the 2.02× over-read affordable rather than fatal: the same bytes cost in-region bandwidth instead of internet bandwidth, and the multiplier stops mattering.

Four benefits beyond speed, the first of which makes §5's dedup design possible:

1. **Extra passes become cheap** — a second read costs ~0.25 h instead of 3.8–19 h (or days, if the CDN figure above is the real one).
2. **Resume never re-hits HuggingFace.** No rate limits, no 429s, no revision drift mid-build.
3. **Reproducibility.** An HF revision can move under us; staged bytes cannot.
4. **It retires the untested path.** The live-HF read stops being a production dependency.

Honest cost: ~$194 and one extra job. Recommended.

3.3 For contrast, the tokenize costs about $40

At the measured 10.5 M tok/s across 32 vCPU and `c7i` on-demand pricing (~$0.04462/vCPU-hour, UNVERIFIED against a live price API), 1.0T tokens is **~$38** — and it is $38 at 32, 64, or 96 vCPU, because the work is fixed and the rate scales. Roughly **$77** with the 2.02× over-read included.

So tokenization is neither the wall-clock bottleneck nor a cost item. That is the context for §7's gigatoken recommendation.

⚠️ **One caveat on that claim, and it is unmeasured.** The 13-gram decontamination scan is roughly **193 billion Python-level `blake2b` calls** and its CPU cost **has never been measured**. §8A attributes essentially all build CPU to tokenization; if the scan is comparable, that attribution is wrong and the build is not as CPU-cheap as this section implies. Worth measuring in the same smoke job.

And there is no Batch timeout to design around. `INGEST-CALIBRATION.md:19-22` corrects this in-repo: the 7200 s figure was *our own setting*, not a platform ceiling. The real constraint is the **128 vCPU compute environment**.

---

4. Sources — the format and encoding traps

Each of these silently produces a wrong corpus rather than an error, which is why they are listed individually.

4.1 DCLM cannot be drawn from the sample, and the full corpus is unreadable

The plan wants **378B tokens**. Three repositories exist and they differ in format:

repo	format	tokens	usable
`HuggingFaceFW/dclm_100BT`	parquet	**114.7B MEASURED**	✅ but only 30% of what is needed
`mlfoundations/dclm-baseline-1.0`	`.jsonl.zst`	~3,764B DERIVED	❌ **unreadable**
`mlfoundations/dclm-baseline-1.0-parquet`	parquet, 27,938 shards	~3,764B DERIVED	✅ **use this**

VERIFIED IN CODE: `READABLE_FORMATS = frozenset({"parquet", "json.gz"})` (`corpus_build.py:127`), and `_assert_readable` rejects anything else **at plan time** (`:171`). `corpus_read.py:774-775` states it directly: "`.zst` *is NOT among them … needs a zstandard dependency this package does not declare.*" `pyproject.toml` declares only `boto3` and `numpy<2.5`.

Two traps in the parquet mirror:

- Its row count is an **estimate**. `/statistics` returns a permanent HTTP 501, and `/size` returns `num_rows=779,982` with `partial:true` — the converted head, **0.03% of the corpus**. A pipeline reading `num_rows` from `/size` sizes this source at **1/3800 of reality**.
- It has **6 columns to the original's 8**, dropping the WARC metadata struct and `warcinfo`. The card calls it "an identical copy"; it is identical in `text`, not in provenance.

4.2 FinePhrase — the nested column, and the trap that does not raise

The rewrite lives at `path_in_schema` `rollout_results.list.element.text`. The top-level `text` column holds the **original FineWeb-Edu document** — its `dataset` field literally reads `HuggingFaceFW/fineweb-edu`.

A flat leaf scan finds `text` **twice**, and `.names.index("text")` returns the original. That builds a corpus of unrephrased web text labelled synthetic, and **no hash, size, or decode check catches it**: the bytes are real text, correctly tokenized, in valid ids.

Worse, the near-miss spelling `rollout_results.text` **does not raise either** — it returns a table with zero columns. Only the exact `path_in_schema` works. Verified through the finished reader; both near-miss spellings are rejected.

4.3 FineWeb-Edu's overlap with FinePhrase is total, not partial

`sample-100BT ⊂ sample-350BT`, and `sample-350BT` is FinePhrase's exact parent. So **100%** of an edu-web draw has a synthetic sibling. The free fix: draw synthetic from the ~242M `sample-350BT` ids **not** in `sample-100BT`.

4.4 Two sources ship a domain, and cardinality is permanent

`stackv2-edu` carries 73 languages via `metadata.gha_language`; `stackexchange` carries ~180 sites via `metadata.site`. **Every distinct value becomes a directory inside** `manifest_sha256`**.** Fold to the top ~20 with the rest as `other`. The fold map is a **reader argument, not a spec field**, because a streaming pass cannot know the top 20 before it has read everything.

4.5 Short documents are a publish blocker, not a preference

`families/pretrain.json` sets `eos_fraction_max: 0.05`, which is a **mean-document floor of 20 tokens**. FinePhrase has a sampled rewrite that is **12 tokens** long. A mean under 20 puts the packed shard's EOS fraction over the bound and **Gate A rejects it — after the tokenize and the upload.** `corpus.MIN_DOC_TOKENS = 64` is what prevents that.

Measured means, for reference — all clear the bound comfortably:

source	tokens/doc	EOS fraction
pubmed	7,917.9	0.000126
peS2o	6,474.0	0.000154
finewiki	1,320.1	0.000758
stackexchange	728.8	0.001372
whole reservoir	**814.9**	**0.001227**

Note `ubuntu-irc` measures 8,650 tokens/doc — its IRC turns are concatenated per document, so it is *not* the short-document risk. FinePhrase is.

---

5. Deduplication

5.1 What the code does today

`dedup_and_decontaminate` drops exact duplicates then contaminated documents, keying on the sha256 of NFC-normalized, right-stripped text. `SeenHashes` stores the **top 128 bits as an int**, not the hex string — a deliberate memory fix, MEASURED with `tracemalloc`: 85.9 B/entry against 154.9 B/entry for `set[str]`.

Scope is per-bundle. `corpus_build.py:482` passes no `seen=`, so `corpus_filter.py:302` allocates a fresh set per bundle. That was a documented choice at 251B — "dedup here is within a bundle, which is where duplicates actually cluster."

5.2 Why it does not scale unchanged

⚠️ **Read 5.2a before this section's bundle table.** The per-bundle figures below are the RESERVOIR mix scaled 3.981×, which is **not** this corpus's mix. 5.2a redoes them against the report's actual shares and reaches a different worst bundle. The *conclusion* — the `set` OOMs, use a flat array — survives both, which is why the section is ordered this way.

Document count is **MEASURED**, not estimated: 308,291,107 documents for 251,218,001,920 tokens = **814.9 tokens/document** (`artifacts/reservoir/realized-tokens.json`, 27 receipts). At 1.0T *scaled from the reservoir* that is **1.23B documents**, with 2.0B as a planning bound if the mix shifts toward synthetic (FinePhrase averages 263–442 tok/doc against pubmed's 7,918 — a 30× spread).

⚠️ **And the current design does not merely miss cross-source duplicates — it runs out of memory.** This is the third blocker, and it appears in no design document.

The Batch container is **14 GiB (15.03 GB)**, sized from a measured worst-bundle resident of ~12.1 GB. The largest bundle *by document count* at 1T is not the largest by tokens: it is `synthetic-finephrase-table--train`, whose mean document is only 263 tokens — 56,839,223 documents at 252B becomes **225.6M at 1T**.

representation	worst bundle @1T	global @1T	fits 15.03 GB?
`set[int]` 128-bit (**current code**)	**19.37 GB**	105.4 GB	**NO**
`set[int]` 64-bit	17.57 GB	95.6 GB	**NO**
flat `np.uint64`, 16 B/key	3.61 GB	19.6 GB	yes
flat `np.uint64`, 8 B/key	**1.80 GB**	**9.8 GB**	**yes**

Dedup is only one resident structure: add ~0.45 GB for the decontamination index, 0.4 GB tokenizer, 0.5 GB pyarrow row group, plus the interpreter — so the usable budget is closer to **13.6 GB**.

Which bundles fail, at the RESERVOIR mix scaled 3.981× (this corrects an earlier draft that named `stackv2-edu` from a stale 155 B/entry figure):

bundle, reservoir mix @1.0T	documents	`set[int]` @85.9 B	verdict against 13.6 GB usable
`synthetic-finephrase-table`	**225.6 M**	**19.37 GB**	**OOM**
`synthetic-finephrase-math`	191.9 M	16.48 GB	**OOM**
`stackv2-edu`	168.1 M	14.44 GB	**OOM**
`synthetic-finephrase-tutorial`	137.3 M	11.79 GB	fits, 1.8 GB of headroom
`synthetic-finephrase-faq`	134.6 M	11.56 GB	fits, 2.0 GB of headroom

Corrected 2026-08-07: an earlier version of this table labelled the last two rows "*over the usable budget*" and `stackv2-edu` "*fits the container, not the budget.*" Both were wrong against this section's own 13.6 GB figure — 11.79 and 11.56 are under it, 14.44 is over it. The three OOM rows are what carries the argument, and they are unaffected.

The FinePhrase bundles dominate because their mean document is **263 tokens** against `pubmed`'s 7,918 — a 30× spread — so they hold the most *documents* despite not holding the most tokens.

5.2a ⚠ But that table is the wrong mix, and the right one moves the worst bundle to DCLM

The figures above are the reservoir's 27 bundles scaled by 3.981×. **The reservoir is 23.82% synthetic; this corpus is 4.3%** (report §4). So the table over-weights exactly the bundles that dominate it. Redone against the report's combined shares, using each source's own **MEASURED** tokens/document from the same 27 receipts:

component @1.0T	B tokens	tok/doc	M documents	`set[int]` @85.9 B	basis for tok/doc
DCLM-baseline	**410.0**	**1,261.5**	**325.0**	**27.92 GB**	MEASURED (`dclm`)
FineWeb-Edu	252.0	1,007.1	250.2	21.49 GB	MEASURED
FinePhrase, one partition	36.0	263.0	136.9	11.76 GB	MEASURED, shortest config
code (`stackv2`)	108.0	943.0	114.5	9.84 GB	MEASURED (`stackv2-edu`)
Nemotron-CC-Math	61.0	1,596.3	38.2	3.28 GB	PROXY (`finemath`)
dolma3 QA	14.0	442.2	31.7	2.72 GB	PROXY (`finephrase-faq`)
everything else (7 rows)	119.0	—	63.6	5.46 GB	mixed
TOTAL	**1,000.0**	**1,041.4**	**960.2**	**82.5 GB**	—

Three things change, and one of them matters a great deal.

1. **The worst single bundle is `DCLM-baseline` at 325M documents / 27.92 GB — not `finephrase-table` at 225.6M / 19.37 GB.** DCLM is 41% of the corpus in one source, and `domain_column` is `None` for it, so **it does not fan out into sub-bundles** — the whole 410B lands in one dedup set. That is **1.44× worse than the figure this plan called the blocker**, and it is a bundle the 5.2 table does not contain at all, because the reservoir drew only 29.8B of DCLM.
2. **The corpus total falls to 0.96B documents, not 1.23B** — the mix is *longer*-documented than the reservoir (1,041 tok/doc against 814.9), because DCLM and FineWeb-Edu displace short synthetic text. So §5.3's 256-way partition is sized on a count that is **28% too high**, which is the safe direction.
3. **The 2.28B figure in `orchestrator-findings.md` F4 is superseded and its own file says so** — it applied the *synthetic* mean (438.5 tok/doc) corpus-wide. F4 carries the correction inline; the Bloom sizing in that finding inherits the error (see §5.3).

Neither the blocker nor the fix changes. A flat `np.uint64` at 8 B/key holds DCLM's 325M documents in **2.60 GB** and the global 960M in **7.68 GB**. The 256-way partition drops to **0.030 GB/worker**. What changes is *which bundle to smoke-test first* — and it is the one nobody had sized.

⚠️ **Two caveats, stated because this table is the basis for a container size.** Four of the thirteen tok/doc values are PROXIES from a different source, marked above. And DCLM's 1,261.5 is measured on the `dclm_100BT` sample as drawn by the reservoir, not on the 3,764B parquet mirror this plan actually reads (§4.1) — a mirror whose row count is an *estimate*. **Measure DCLM's real tokens/document during the Phase 2 smoke test**, where the reader is already running.

Corroborated by a real incident, not just arithmetic. At 251B this already bit: the dedup set was measured at **155 B/entry** before the int narrowing (its docstring had claimed 113 B, a 37% understatement), and that "*is what pinned all four hosts at 97% memory with 25% of their CPU idle.*" The narrowing to 85.9 B bought headroom at 251B; at 1.0T it is not enough.

VERIFIED MYSELF, and it corrects a natural intuition: narrowing the key *inside* a Python `set` does not help. `sys.getsizeof` gives 44 B for a 128-bit int and 36 B for a 64-bit one, so against the codebase's measured 85.9 B/entry the set's own slot overhead is **41.9 B/entry and width-independent** (CPython stores a pointer plus a cached hash, not the value). Narrowing 128→64 buys **9.3%, not 50%**.

The 5–11× win comes entirely from **leaving the `set` for a flat numpy array**.

5.3 Proposed: a sort-based dedup pre-pass, partitioned 256 ways

Two independent constraints point at the same design, which is what makes it the right one rather than merely the cheapest.

Constraint 1 — memory. A flat `np.uint64` accumulator plus `np.unique` is 8 B/key: **1.80 GB for the worst bundle**, 11× denser than the current set. But a flat array cannot answer "seen?" incrementally, so it forces a separate pass.

Constraint 2 — determinism. A shared mutable filter threaded through the build would make the output depend on bundle execution order, destroying the byte-identical-rerun property that §1 shows is already verified. A pre-pass that emits an immutable keep-list preserves it.

The design. Pass 1 reads the staged text and emits `(hash, source, ref)` triples. The hash space is partitioned 256 ways; each worker owns partition `p` and therefore sees documents from **every** source whose hash falls in `p`. Cross-source duplicates land in the same partition by construction, so the result is **exact global dedup with zero shared state** — no Bloom false positives discarding real documents, no coordination. Sort, unique, resolve winners by an explicit source priority (§5.5), emit a keep-list. Pass 2 builds shards against it.

Sized at **1.23B documents** — the reservoir-scaled count, deliberately kept as the planning figure even though §5.2a measures the real mix at **0.96B**, because over-provisioning a partition count is free and the mix can still shift:

partitions	resident per worker @1.23B docs (planning)	@0.96B docs (§5.2a measured mix)
64	1.65 GB	1.28 GB
128	0.82 GB	0.64 GB
256	**0.41 GB**	**0.32 GB**

⚠️ **Those columns are 16 B/key** — a `(hash, ref)` pair held during the sort — not the 8 B/key of the accumulator in §5.2's table. Both numbers are correct for different structures, and confusing them is easy: the 8 B figure sizes "*can I hold every hash?*", the 16 B figure sizes "*can one worker sort its partition?*"

Triple volume is 1.23B × 26 B = **32 GB**, trivially sortable in S3. **The second pass is only affordable because of §3's staging** — without it, a second read costs another 2.7–5.4 h from HuggingFace instead of ~0.2 h from S3.

Why not a Bloom filter. At n=1.23B and fp=1e-4 it is only 2.94 GB and fits the existing container, which is genuinely attractive. But false positives **discard real documents** — 123K documents ≈ 0.10B tokens at that rate — and it cannot be made deterministic as a shared mutable structure. Keep it as the fallback if the pre-pass proves operationally awkward, and record the 0.01% loss explicitly if so.

⚠️ `orchestrator-findings.md` **F4 reaches the opposite verdict — "the only affordable global option" — and it is superseded.** Two things separate that 8.20 GB from this 2.94 GB, and neither is a disagreement about Bloom filters:

	F4	here
documents	2.28B (the **synthetic** 438.5 tok/doc applied corpus-wide)	1.23B planning / 0.96B measured (§5.2a)
target false-positive rate	1e-6	1e-4
size	8.20 GB	2.94 GB

F4's document count is corrected inside its own file. And F4 called Bloom "*the only affordable global option*" because it was comparing against a **global** `set` at 195.9 GB — it had not yet considered the flat-array pre-pass, which is 7.68 GB global at the measured count and **exact**. **A Bloom filter is the fallback, not the plan.** At n=0.96B and fp=1e-4 it would be 2.30 GB.

The 8-byte truncation is safe. Birthday collision probability over 1.23B documents at 64 bits is **4.08%** — but that is the probability that *any* collision exists at all; the expected number of colliding pairs is **0.04**, i.e. expected data loss under one document. The current 128-bit width is over-provisioned by 64 bits.

5.4 What exact hashing cannot do, and why that is mostly fine

Near-duplicates survive: boilerplate-differing pages, the same article on two domains, a document differing by one interior whitespace character. Every major 2026 corpus uses MinHash/LSH instead.

Two pieces of counter-evidence say not to reach for it reflexively. FineWeb measured that **global cross-dump MinHash was actively harmful** — the removed data outperformed the kept data. And the often-cited −11.8 MMLU from DCLM Table 19 is a **deduplication** result, not a decontamination one. Meanwhile our upstream sources have each already run their own dedup, so our remaining job is **cross-source only** — a much smaller claim than "we need MinHash."

Recommendation: exact, sharded, global. Do not add MinHash for this build. Record the decision so it can be revisited with a measurement rather than a reflex.

5.5 Ordering is an unrecorded decision

`dedup_and_decontaminate` keeps the **first** occurrence, and bundles run in registry order — so which source "wins" a cross-source duplicate is decided by alphabetical accident. Make it explicit: an ordered source-priority list, highest-quality-source-wins, recorded in the plan.

5.6 The receipt does not record what was removed

`run_bundle` returns `filter_stats.as_dict()` and `_cmd_run` prints it, but `Receipt` **has no filter block** — grep for `duplicates` or `contaminated` in `corpus_receipt.py` returns zero hits. The real duplicate and contamination rates for our own sources exist only in CloudWatch logs from the 2026-08-05 run. Cheap to fix, and until it is fixed no quantitative dedup claim is auditable.

---

6. Decontamination

6.1 What the index holds, verified item by item

`DecontaminationIndex` runs two independent tests, OR'd: the exact content hash, or `minimum_hits=2` distinct 13-gram hits. The shipped index is **149,777 exact hashes + 3,097,372 13-grams**, ~250 MB resident — budget it on Batch, it is not free.

Coverage, MEASURED against the manifest's own `source_counts` (127 keys, 148,458 base-unique texts):

benchmark	present	splits	items
MMLU	✅	all 57 subjects × {validation, test}	62,102
GSM8K	✅	`main/test` only	1,319
ARC-Challenge	✅	test + val	4,687 + 1,194
ARC-Easy	✅	test + val	9,496 + 2,281
HellaSwag	✅	`val` only (test labels are unreleased)	40,145

All four benchmarks we steer on are present, plus seven bonus families. Good.

6.2 ⚠️ But the index is built from 5-shot RENDERED prompts, which kills half of it

The single highest value-per-dollar fix in this review. `eval_bundle.py:141–170`: each MMLU entry is a subject preamble + **five worked demonstration Q/A pairs** + the target question + **one answer choice**.

Consequence 1 — the 149,777 exact hashes are effectively dead for MMLU, ARC and HellaSwag. An exact hash fires only on a corpus document byte-identical to a full OLMES render with one specific choice appended. Real contamination is a bare question, or a question with its options laid out differently. It never looks like that render.

GSM8K is the exception, and it explains the evidence. Its entries are `question + "\n" + answer` — a shape a web page can genuinely match. Which is exactly why the one real-index test that passes is the GSM8K one: "*40/40 GSM8K test questions caught*." There is no MMLU equivalent, and now we know why.

Consequence 2 — the n-gram half survives, but diluted 21×. VERIFIED: 3,097,372 distinct 13-grams over 148,458 unique texts is **20.9 per text**, against ~438 windows a 400–500 word render would naively yield. The collapse is the set de-duplicating the shared preamble and five demonstrations across ~1,090 items per subject — so what survives as distinct entries are the windows **spanning the target question and its choice**. The question does get indexed, at ~21 windows rather than ~438.

Consequence 3 — the short-item hole is on the index side too, and it is now quantified. A benchmark question under ~14 words yields windows only in combination with template words (`Question:`, `Answer:`, the previous demonstration's tail), so it can match only a document reproducing the template. Worse than the usual framing, which notes only that short *documents* yield zero windows.

The bound, from GPT-3 Table C.1: 5th-percentile benchmark item lengths are **11 / 12 / 13 words** across the three suites. So **at least 5% of the items in the two 11-word and 12-word suites are shorter than a single 13-gram** and yield **zero** windows — reachable only by the exact-hash half, which Consequence 1 shows is inert for exactly those suites. Both halves fail on the same items.

⚠️ **Precisely:** an item of exactly **13 words is exactly one 13-gram**, not shorter than one — it yields a single window, which cannot reach `minimum_hits = 2` and so is **equally undetectable**, by a different mechanism. So the third suite is not evidence for "shorter than one 13-gram," but it lands in the same place: `minimum_hits = 2` needs **≥14 words** to be satisfiable at all. An earlier draft stated the bound as "≥5% of ARC/HellaSwag items are shorter than one 13-gram," which is right for the 11- and 12-word suites and imprecise for the 13-word one.

That makes this **corpus-corrupting for the short-item suites specifically**, not merely a theoretical gap — and it strengthens the case rather than weakening it, because the `minimum_hits` floor extends the dead zone from "under 13 words" to "under 14."

FIX: rebuild the index from raw benchmark fields — question alone, question + each choice, question + correct answer — **in addition to** the rendered form. One CPU job, no GPU, using the pinned `ai2-olmo` checkout.

DERIVED cost: a bare ~30-word MMLU question yields ~18 windows × 62,102 items ≈ **1.1M new n-grams.**
⚠️ **The two percentages that follow have different denominators and an earlier draft stated them as one figure:**

	against what	value
n-gram **count**	+1.1M on 3,097,372 existing	**+36%**
resident bytes	+18 MB on the ~250 MB index	**+7%**

Both are correct; neither is "+36% and +18 MB" as a single claim. The bytes grow more slowly than the count because the exact-hash half does not grow at all. **Cheap either way, and it restores the exact-hash half.**

6.2a The n-gram size: keep 13, and do not relitigate it without a measurement

The design doc specifies `allenai/decon` at `ngram_size 5, stride 10, threshold 0.8`; the code shipped a reimplementation at **13-gram with** `minimum_hits = 2`. Two independent audits in this review reached **opposite verdicts** on that divergence, so here is the reconciliation.

What the two configurations are, since this section refers to them as "13/2" and "5/0.8" throughout:

	shipped	specified
n-gram size	**13**	**5**
trigger	`minimum_hits = 2` — two distinct n-gram hits, an absolute count	`threshold 0.8` — the fraction of a benchmark item's n-grams that must match
stride	1 (every window)	10
match weighting	none	IDF, plus cluster expansion

So `0.8` **is a coverage fraction, not a second n-gram size** — it means "*flag the document when 80% of some benchmark item's 5-grams appear in it.*" The two rules are not comparable by inspection: 13/2 fires on two long coincidences anywhere, 5/0.8 fires on broad shallow coverage of one item. That is *why* nobody can call the winner from reading the code.

Both agree DCLM's famous −11.8 MMLU is a deduplication result, not a decontamination one. From v4 Table 19: `min_ngram` 5 gives MMLU **32.5**, `min_ngram` 13 gives **44.3**, while Core moves only 45.3 → 44.5. The two rows also differ in shard count, so it is **not a clean single-variable ablation**.

The argument for keeping 13 is a mechanism, not the number. Short-n-gram collision causes mass removal of MMLU-relevant material, and that mechanism is identical whether the match set is other corpus documents (dedup) or benchmark items (decontamination). Decontamination is arguably *worse*: its match set is deliberately concentrated on exactly the knowledge MMLU tests, so the collisions are **aimed at the metric** rather than random with respect to it. Independent corroboration: non-member **7-gram** overlap has been measured at **32.5–77%** depending on domain; 5-gram collision is higher.

The counter-argument, which is real: `allenai/decon` at 5-gram weights matches by **IDF** and requires **cluster expansion**, which suppresses precisely the common-phrase collisions that would sink a naive 5-gram filter. So the design doc's 5-gram is **not the same object** as DCLM's 5-gram row. That difference may well be sufficient — **but nobody has measured it**, and adopting it bets the project's headline metric on an untested assumption.

Decision: keep 13/2 for this build and document the divergence. The asymmetry decides it — a false negative leaves one benchmark item in a 1T corpus, while a false positive at 5-gram risks the mechanism that

cost DCLM 11.8 MMLU. **Nobody has measured 13/2 against 5/0.8 on our corpus; that is a finding, not a gap.** Revisit only with that measurement.

⚠️ **And fix §6.2 first regardless.** The rendered-prompt defect is a defect at *any* n-gram size, so tuning `n` before rebuilding the index is tuning the wrong parameter.

6.3 Three coverage gaps worth naming

- **MATH, HumanEval, MBPP, DROP, TriviaQA, BBH, MMLU-Pro and GPQA are entirely undefended.** The index covers the 9 OLMo-ladder families only. This mix carries substantial math and code, so if the program ever reports MATH or HumanEval, those numbers have **no contamination defense at all** — while the build prints a healthy-looking `DECON index 149,777 exact + 3,097,372 ngrams` either way. **This is the exact failure mode that matters: an index missing a benchmark we steer on reports success.** Decide the eval suite, then extend the index to match it.
- **GSM8K `train` is absent** (test only). Leakage of GSM8K *train* is the documented failure mode — Nemotron STEM-SFT was seeded from GSM8K/MATH/AOPS train splits. It does not directly inflate a test score, so severity is low, but it becomes real if train items are ever used as few-shot demonstrations.
- **MMLU `dev` is present only as the embedded 5-shot preamble**, not as items. The docstring claiming `{dev, validation, test}` coverage is imprecise and should be corrected.

6.4 The rephrasing hole, which nothing here closes

FinePhrase is a rephrasing of FineWeb-Edu, and **n-gram decontamination is defeated by rephrasing by construction**. If a benchmark item leaked into FineWeb-Edu, its FinePhrase rewrite carries the same knowledge with zero shared 13-grams. Our own measurement of the general case: 13-gram detection scores **F1 0.926 on verbatim text and 0.000 on rephrased text.**

There is no affordable defense. Embedding- or model-based detection over 1.23B documents is not proportionate here. **The honest position is to accept it and disclose it in `limitations[]`** — and to note that HellaSwag is where contamination persists longest while measuring 0.00% n-gram-dirty, which makes it simultaneously our least-served and least-instrumented metric.

6.5 The tier-1 exclusions

Nobody earns a decontamination TRUST verdict across the 17 audited corpora. Exclude 9 items at source — **under 1% of tokens, carrying nearly all of the risk.** The concrete one already identified: `data_provenance_initiative` ships `fc-cot-cot_gsm8k` (GSM8K in Flan CoT format) at 6 repeats with a ~9× cooldown upweight, and costs **0.51% of tokens** to drop.

---

7. Tokenization

7.1 The contract, which holds whatever tokenizer implements it

Four properties the build depends on. A replacement tokenizer must preserve all four.

1. `add_special_tokens=False`**, and we append EOS = 100257 ourselves.** The library default is `True`, and what it then adds depends on a post-processor nobody in this repo owns. Appending it ourselves makes `1 / mean_doc_tokens` a number the packer can assert — which is the entire basis of the EOS gate. **For**

dolma2 this is moot in the best way: `post_processor` **is** `null`, so `add_special_tokens` is a no-op in either direction (MEASURED — read the live `tokenizer.json`).

2. **Every id asserted into** `[0, vocab_size)` **before the uint32 cast.** The cast cannot fail: MEASURED on numpy 2.4.4, assigning `np.array([-1, 5], dtype=int64)` into a `<u4` buffer yields `[4294967295, 5]`, and `2**33` yields `0` — both silently. The assertion is load-bearing.
3. `vocab_size` **includes added tokens.** dolma2 is 100,278 real / 100,352 padded, with 22 added tokens at ids 100256–100277. Because `eos_token_id` 100257 lives inside that range, **a tokenizer shipping an empty** `added_tokens` **yields** `eos_token_id: None`**, and Gate A's EOS check then skips silently** — the live defect in task #12, present in two already-published corpora.
4. **EOS is appended at tokenize time, not pack time.** So re-packing at a different `seq_len` requires **re-tokenizing**. A deliberate trade: the EOS is the only document boundary this corpus will ever have, so it belongs inside the unit whose contract guarantees it.

7.2 The tokenizer stays dolma2

No alternative clears a measured downstream gain. Two facts make this more than inertia: `allenai/dolma3-tokenizer` **is** dolma2 (AI2 says so in their own code), and AI2's pre-tokenized shards are **byte-identical to our** `.u32le.bin` — verified by range-read, no NumPy header, valid ids from byte 0. Their 5.93T corpus is therefore a byte copy for us, not a re-tokenization.

7.3 gigatoken — audited, and safer than expected

`github.com/marcelroed/gigatoken`, MIT, Rust with Python bindings, CPU-only. Claims ~1000× over HuggingFace `tokenizers`. Since our corpus is content-addressed and a wrong-but-in-range token id is invisible to every gate we own, **exact output parity was the only question that mattered**; speed was secondary.

What the audit established, all MEASURED by reading source:

- **The pretokenizer regex is byte-identical.** dolma2's `Split` pattern and gigatoken's `OLMO3_REF_REGEX` in `src/pretokenize/fast/olmo3.rs:261` are **the same 115 characters**. The internal name "olmo3" covers OLMo 2 and 3, and dolma2 is exactly that scheme — the README's "OLMo 2/3" row is not a lookalike.
- **The repo's own parity fixture is semantically our tokenizer.** It tests `allenai/Olmo-3-1025-7B`, whose `tokenizer.json` differs from `allenai/dolma2-tokenizer` **only in merges serialization** — same 100,278-entry vocab dict, same 100,000 merges, same pretokenizer, same 22 added tokens, same EOS id. So every existing ID-parity assertion already covers our tokenizer's behaviour.
- **There is no lossy fast mode.** The README's "compatibility vs native" split is an **API-shape** distinction, not a lossy-vs-exact one: `_hf_compat.py` calls the same native Rust backend and then builds transformers-shaped Python objects. Both tiers produce ids from one encoder.
- **We are not forced onto the slow wrapper.** The native `encode_batch` accepts `list[str] | list[bytes]` directly and fans out over rayon — which is exactly the shape our documents arrive in. `encode_batch_list` returns `list[list[int]]` assembled in Rust, avoiding an `awkward` dependency.
- **An unrecognized pretokenizer is a hard error, not a silent wrong split.** Scheme selection is an exact string match with `None` converted to `Err`. This was the scariest hypothesis in the brief and the source refutes it.
- **The legacy-merges risk was raised and then closed by measurement.** `allenai/dolma2-tokenizer` ships the old space-joined form (`'Ġ Ġ'`) that the tested fixture does not. `hf.rs:113-137` accepts both via

a `serde(untagged)` enum **over the whole list**, so a partial parse is not representable — either every entry deserializes or the load fails. Verified on our actual file: all 100,000 entries are legacy-form, **0** have a space count other than 1, and for all of them both halves *and* their concatenation resolve in the vocab. The legacy parse reconstructs the merge table exactly.

- **The differential suite is real, and unusually good** — 8 tokenizers including olmo3, 36 adversarial texts (emoji, CJK, RTL, non-NFC decomposed, `"a"*500`), 28 special-token texts including the exact dolma2 added-token set plus a lookalike battery that must *not* match, and `tests/test_encode_dclm.py` runs exact ID parity on **DCLM-baseline documents selected for tokenizer-hostile content** — literally our largest source. **But CI does not run it.**

The speed question, calibrated against our own measurement rather than the README. Our build measures **10.5 M tok/s** with HF `encode_batch` across 32 vCPU (MEASURED), so 1.0T tokens is **26.5 h**. The audit projects **20–60× for our workload**, discounting the README 4× for heterogeneous data, no AVX-512, and Python overhead — putting tokenization at **0.5–1.5 h**.

But tokenization is not the bottleneck, and that is the crux:

stage	today	with gigatoken
read ~6.1 TB of source	~13.5–27 h	**unchanged**
tokenize 1.0T	26.5 h	~0.5–1.5 h
total, serial	~40–53 h	~14–28 h
total, overlapped	~26.5 h (tokenize-bound)	~13.5–27 h (**download**-bound)

So in an overlapped design the win is **0 to ~13 h**, depending on a number we have not measured: our sustained in-region S3 read bandwidth. Our own memory records `publish()` pulling at 0.8 MiB/s *out of region*, which is why in-region rates need measuring rather than assuming.

The real argument for it is not wall-clock — it is retry cost. A 26.5 h tokenize needs `attemptDurationSeconds` far past the default and is expensive to re-run after a bug. A 1 h tokenize is cheap to re-run, which changes how freely the mixture can be iterated. Given this project's history of finding corpus defects *after* publishing, cheap re-tokenization has option value beyond this build.

7.4 Three findings that force a gate rather than a straight adoption

- **CI runs no tests.** `.github/workflows/CI.yml` is stock maturin — it builds wheels and publishes to PyPI on tag. No `pytest`, no `cargo test`. The good differential suite above is **developer discipline, not enforcement**, which means *a version number is not evidence of parity*.
- **Unicode-version divergence, declared WONTFIX.** gigatoken resolves `\p{L}` / `\p{N}` through ICU4X 2.2 (Unicode 17); HuggingFace `tokenizers` 0.20.x uses Oniguruma (Unicode 14). The maintainer closed the report as WONTFIX. So the two can disagree on recently-assigned codepoints **by design**.
- **A confessed prior bug in exactly our risk shape.** `fast/mod.rs:129–155` documents a fixed defect where pretokens over 65 KB of invalid UTF-8 split **nondeterministically** between code paths. And a related issue's symptom was *same token count, different order* — invisible to every check we own.
- **No fuzzing exists** (zero hits for fuzz/proptest/quickcheck). Testing is differential over fixed corpora: real and three-level, but not generative.

7.5 Recommendation

Do not put gigatoken on the critical path for this build. Keep HF `tokenizers` — it built the 251.2B corpus and every test exercises it. The reasoning is not that the risk is large; it is that **the benefit is currently unmeasured while the risk is not zero**, and tokenization is not the bottleneck.

Measure first, and it is one cheap job: our sustained **in-region S3 read bandwidth**. That single number decides whether tokenization is on the critical path at all. Our own memory records `publish()` pulling at 0.8 MiB/s *out of region*, so in-region rates must be measured, not assumed.

If the gate is worth running, this is it — specific enough to submit without a follow-up question:

> Point the repo's own `tests/tokenizers/test_hf_parity.py` fixture at `allenai/dolma2-tokenizer` instead of `allenai/Olmo-3-1025-7B`, and run it together with `tests/test_encode_dclm.py` and `test_owt_matches_hf` at `OWT_MAX_BYTES=0` (the full ~12 GB). **Pass = exact `np.array_equal` on ids for every document, plus `get_vocab_size()` reporting 100,278.** Redirecting the fixture is the required step, not an optional one: it is the only thing that actually **executes** the legacy space-joined merges path, which §7.3's proof only argued for statically. On FarmShare or AWS, never locally.

What makes the residual risk acceptable rather than merely small: tokenizer parity is **run-once-then-frozen**. The corpus is immutable, so one job's evidence covers its entire life.

The honest limit of that gate, stated because it undercuts the recommendation: the Unicode trigger set is a few thousand codepoints, so a one-million-document gate can come back clean while a run a thousand times larger still diverges on a few thousand documents. A clean gate proves a **low rate, not zero**. The only complete check is the full 26.5 h HF run — which destroys the reason to adopt in the first place.

It flips for the next build. There, re-tokenization is the *point* rather than a cost: the ~$600 SuperBPE A/B needs multiple passes over the same text, and the download is already paid for by the staged copy in §3.

Two supporting reasons this ordering is right regardless: the corpus is content-addressed, so a divergence found after publishing means a full re-copy rather than a patch; and swapping the tokenizer changes nothing about the blockers in §0, which is where the risk actually lives.

7.6 ⚠️ The tokenizer implementation we already use is unpinned

Found while checking the above, and it matters more than the gigatoken question.

`tokenizers` **appears nowhere in** `pyproject.toml`. Declared dependencies are exactly `boto3` and `numpy<2.5`, while `corpus_build.py:631` does `from tokenizers import Tokenizer` and `tokenize_documents` calls `encode_batch` on it. So **the package that decides every token id in the corpus is resolved at container-build time to whatever PyPI served that morning.**

This is precisely the hazard that file's own `numpy<2.5` comment exists to prevent — and it applies with more force here, because a numpy mismatch crashes while a tokenizer change **silently emits different ids that stay in range and decode**. The Unicode-table axis above makes it concrete rather than theoretical: two builds of the same corpus on different mornings can differ on documents containing recently-assigned codepoints, and no gate we own can tell.

This environment has 0.22.2, so we are not on the flagged 0.20.x generation locally — but production is unconstrained.

Fix before the first ingest job: declare and pin `tokenizers`, and record the resolved version in the receipt beside `wheel_version`, so a corpus can name the implementation that produced it and not merely the vocabulary it used.

---

8. Validation and publish topology

8.1 Gate A costs 8 network round trips per object, and 7 of them are serial

The table below accounts for six of the eight; see the correction under it for the other two. MEASURED-IN-CODE, per manifest entry:

check	call	threaded?
Gate A decision loop	`s3.head` — real size vs declared `bytes` (`validate.py:703`)	**yes**, `--head-workers`
`_observed_size`	`s3.head` , cached per key (`pretrain_tokens_v1.py:222`)	**no**
`check_decode_smoke`	4 × `s3.get_range` at seeded offsets, 16 KB each (`:240`)	**no**
`check_first_bytes_not_npy`	`s3.get_range(key, 0, 8)` — the `\x93NUMPY` sniff (`:438`)	**no**
`check_seq_len_alignment`	cache hit, free	n/a

The 85-minute figure everyone quotes is **MEASURED live** and recorded at `pretrain_tokens_v1.py:205–210`: "*Gate A ran ~85 min at 0.3% CPU and ~15.8 round trips/s — purely latency-bound, which is what pushed the first promotion attempt past its 2 h timeout.*"

⚠️ **The table above undercounts by two, and the measurement says so.** At 6 calls/object, 10,049 objects over 85 minutes is 11.8 rt/s — but the recorded rate is **15.8**. I originally attributed that gap to per-group LISTs and manifest GETs. It isn't: **8 calls/object gives 15.76 rt/s**, which reproduces the measurement almost exactly, so the "gap" I explained away *was* the two calls I had missed. A model that needs a hand-waved remainder to match a measurement is not yet a model.

So Gate A is `objects × 8 × latency`, and the CPU is idle — which makes every figure below ~33% worse than a 6-call model predicts. The wall-clock in §8.2 is scaled from the measured 507.5 ms/object directly, so **those numbers are unaffected**; only the per-call attribution changes.

8.2 It does not fit any plausible timeout

Shard size is settled: `SHARD_TOKENS = 25,001,984` (`corpus.py:89` = 3052 × 8192), giving ~40,001 objects at 1.0T. Everything below, and every figure in §8A and §9, is computed at that size. An earlier draft of `FINAL-DATASET-REPORT.md` §11 said 50,003,968 → ~20,000 objects; that value appeared in no code and is withdrawn. The 20,000 row is kept here **only** to show what halving the object count would buy, because that is the one real argument for revisiting the constant.

objects	round trips	serial	with 16 head workers
10,049 (**measured**)	80,392	**85 min**	~71 min
20,000 (*hypothetical* 50M shard)	160,000	**2.82 h**	~2.47 h
40,001 (the real size)	**320,008**	**5.63 h**	~4.93 h

Round trips are `objects × 8`; the wall-clock column is scaled from the **measured 507.5 ms/object**, so it is independent of the call count and unaffected by the §8.1 correction. The 10,049 row's 80,392 is not arithmetic — it is the **measured** count from the call-counting spy in commit `db437b6`, and 80,392 ÷ 10,049 = exactly 8, which is what fixed the call count in §8.1.

`--head-workers` **alone barely helps, and the reason is Amdahl's law:** it threads exactly one of **eight** calls, so even at infinite head workers the seven serial calls still cost **87.5%** of the original time. The CLI help is accurate but narrowly scoped — it describes *the decision loop*, while seven of the eight calls live in the profile checks that run afterwards.

⚠️ **The "1-hour validate cap" is UNVERIFIED and should not be planned against as fact.** What the repo actually records: `edullm-validator:7` was registered at `attemptDurationSeconds = 7200` (2 h), revisions 1–6 had `timeout: null`, and the live revision is **12 with its timeout recorded nowhere**. The 1-hour figures in `docs/PLATFORM-INTEGRATION.md` are `attemptDurationSeconds: 3600` on **platform GPU/smoke** job definitions, not the dataset validator. **Get the live number before submitting:** `aws batch describe-job-definitions --job-definition-name edullm-validator`. Either way, 1 h fails and 2 h fails; the honest requirement at default settings is **≥ 4 h**.

Promotion, by contrast, is already solved. It is ~2 round trips per object but **is** threaded on both phases, so 40,001 objects at 16 workers is ~20–30 min.

8.3 The fix, in order of leverage

1. **Raise `max_pool_connections`.** `s3.py:196` builds `boto3.client("s3")` with **no `Config` at all**, so it inherits botocore's default of **10**. `validate.py:2477` documents this for another path. **Threading against a 10-connection pool self-throttles**, so this must come first or the next step underdelivers.
2. **Thread the five profile-check calls.** `validate.py:577` already does exactly this pattern for the size sweep; `profiles/pretrain_tokens_v1.py` has **zero** threading. At 16 workers, 40,000 objects drops from 5.63 h to roughly **21 minutes**. This is task #10, and it is the right fix.
3. **Drop the redundant HEAD.** The npy sniff and the decode windows both need the object size, and a ranged GET already returns it in `Content-Range` — so the cached HEAD can go, taking 8 calls to 7. Smaller win, but it is pure subtraction. **Already partly done and measured:** a call-counting spy recorded **80,392 round trips before the HEAD-cache fix and 70,343 after (−12%)**, which is where the 8-calls-per-object figure comes from.

Once this is fixed, shard size stops being forced by the validator and can be chosen on mixture error and OLMo-core's read pattern — which is the correct basis. Mixture error at 25,001,984 tokens is **0.007%–0.278%** across the stage-1 sources, so it does not constrain the choice either. (The report's former **0.33%** is the *same quantity computed at the withdrawn 50M shard size* — not an independent measurement that disagrees. Both are far below any level that would matter, which is precisely why the constant should be decided on Gate A cost.)

8.4 Publish as TWO datasets, not one

`build_mixture` is scoped to exactly **one group of one dataset**, enforced in three places. And `PATH_LABEL_KEYS = ("source", "domain")` is exactly two levels deep, with `labels_from_path` raising on anything deeper — so "**stage**" **cannot be a third path segment.** Three options, and the recommendation is unambiguous:

option	verdict
one dataset, stage fused into `source` (`dclm-s1`, `dclm-s2`)	works, but doubles the source vocabulary and makes every per-source predicate awkward
one dataset, trainer stages it	defers a decision the corpus should record
two datasets	**recommended**

Why two datasets is right, and it is not just tidiness:

- **A cooldown is sequential by definition**, so nothing ever needs one mixture spanning both stages — which is the only thing the single-dataset constraint forbids.
- **Each validates separately.** Stage 2 at 4,000 objects is **0.56 h serial — it fits today, with no code change.** That turns one impossible validate into one easy one plus one that needs the §8.3 fix.
- **Ordinal blast radius is contained per stage.** The §0 renaming hazard stops crossing between them.
- **Stage 2 can be rebuilt or reweighted without touching stage 1** — and stage 2 is where the QA and reasoning shares are least certain, so it is exactly the half most likely to be rebuilt.

8.5 Interleaving is trainer-side. Definitively.

Task #15 wants micro-batches that are not domain-pure. **It cannot be done in the data**, and here is the proof: `labels` is a field of `ManifestEntry` — **one dict per shard path** — derived from the path segments, and Gate A recomputes it and compares by **full dict equality**. An interleaved shard holds documents from N sources but can carry only one `source` label, so it would be either mislabelled or rejected.

So the fix belongs in the trainer's data loader, and the setting is already identified: `MoELoadBalancingLossGranularity`, which defaults to `local_batch` in four places. Worth **0.13–0.18 perplexity and +5–6 GSM8K**, and it **cannot be annealed in** — switching at 10% of training recovers only ~55%. With only 2 shared experts of 6 active, routing quality is load-bearing rather than incidental.

8A. Wall-clock — and why the same build takes 10 hours or 36

Every figure below is anchored on the **reservoir's real 2026-08-05 run** (27 bundles, 10,049 shards, 251.2B tokens, read from receipts), scaled by **3.981×** to reach 1.0T. That scaling lands on **40,001 shards** and **1,227,185,570 documents**, which is where §5's planning document count comes from.

⚠️ **The 3.981× scaling assumes this corpus has the reservoir's composition, and it does not** — the reservoir is 23.82% synthetic against this corpus's 4.3%. §5.2a redoes the document count against the real mix and gets **960M, not 1.23B**. **Shard count is unaffected** (40,001 shards is tokens ÷ `SHARD_TOKENS`, independent of document length), and so is every duration below, because they are anchored on tokens and

bytes rather than documents. The document count matters only to §5's memory sizing, where it is used in the conservative direction.

8A.1 ⚠️ Read this before trusting any number in this section

The repo's most authoritative-looking timing document, `artifacts/reservoir/INGEST-CALIBRATION.md`, is **a case study in getting this exact task wrong.** It measured a correct number — 0.44 files/s at 16 workers — and drew two confident conclusions from it, both since retracted in its own banner:

- "*The binding constraint is requests per IP.*" **False.** It was **our own 70× quota amplification**: `_RangeFile` sent every one of pyarrow's ~70 per-file range reads to the *metered* resolver instead of resolving the signed **CDN** URL once and reusing it. After the fix, the same pass took **67 seconds** against a **16.9 h** projection.
- "*The 7200 s job-definition timeout*" as a wall to design around. **It was our own setting.** AWS publishes no maximum Batch timeout.

It also carries a self-flagged **16× unit error** (wall-seconds confused with worker-seconds per file) that propagated into `RUN-THE-INGEST.md:35`. **Do not size anything from that file's tables.** Its own closing lesson is the right one: *a throughput measurement that does not record what limited it invites exactly that error.*

8A.2 The measured anchors

#	anchor	value	source
A1	tokenize, `encode_batch`	**10.5 M tok/s across 32 vCPU** (0.328 M/vCPU)	`corpus_pack.py:230-250`
A2	deep re-hash, single stream	**87.8 MB/s**; **7.82× at 8 workers**	`verify-job.json`, `PUBLISH-SPEC.md:167`
A3	Gate A per object	**507.5 ms serial** = **8** round trips, 0.3% CPU	`pretrain_tokens_v1.py:205-210`
A4	promote	~2 round trips/object, **already threaded**	`validate.py:1943-1948`
A5	id/fetch pass, post-fix	**67 s** (was projected 16.9 h)	`INGEST-CALIBRATION.md` banner

8A.3 The one hard floor: 128 vCPU

The compute environment caps at **128 vCPU on one `c7i.8xlarge` type**, and that cap is real (unlike the timeout). Tokenization is CPU-bound, so at A1's measured per-vCPU rate:

vCPU	rate	1.0T tokenize
32	10.5 M tok/s	26.46 h
64	21.0 M tok/s	13.23 h
128 (the cap)	**42.0 M tok/s**	**6.61 h**

6.61 h is the tokenize floor for 1.0T no matter how bundles are sliced. Slicing does not lower it — it only prevents a long tail.

8A.4 Per-stage wall-clock, as-configured versus fixed

stage	as-configured	fixed	what changes it
Pass 0 — stage 4.21 TB to S3	3.0 h	**0.5 h**	parallel copy children
Pass 1 — dedup pre-pass (hash only)	1.0 h	**0.3 h**	reads staged text in-region
Pass 2 — build (read+filter+tokenize+pack)	11.2 h	**6.6 h**	file-shard the big bundles; **6.6 h is the floor**
Publish both stages (40,001 objects)	2.0 h	**0.3 h**	`copy_workers` / `hash_workers` > 1
Gate A validate	**5.6 h** ❌	**0.36 h**	thread the profile checks + raise the pool
`verify --deep`	**13.0 h** ❌	**1.66 h**	`--hash-workers 8` (**measured 7.82×**)
promote	0.2 h	0.02 h	already threaded
TOTAL	**~36 h**	**~10 h**	

Both ❌ rows fail outright at 1.0T, not merely run slowly: Gate A's 5.6 h and `verify --deep`'s 13.0 h each exceed their job timeouts, so the corpus could not be promoted at all.

⚠️ **Gate A appears three times in this plan at three values. All three are the same rate; only the denominator differs.** Naming them, because an earlier draft did not:

where	objects	serial	note
§8.2	40,001 — **both stages**	**5.63 h**	the arithmetic table
§8A.4 above	40,001 — both stages	**5.6 h**	same figure, rounded
§9 Phase 4	**36,000 — stage 1 only**	**5.08 h**	stage 2's 4,000 objects are a separate 0.56 h job

All three are `objects × 507.5 ms`. **Quote the one whose object count matches what you are validating** — the two-dataset topology of §8.4 means no single job ever validates all 40,001.

8A.5 The critical path is one bundle, and it is CPU

Wall-clock per child is **read + tokenize serialized, not overlapped** — `corpus_read`, `corpus_build`, `corpus_pack`, `corpus_filter` and `s3.py` contain **zero** threading (grep-verified), so a child alternates between fetching and encoding in one generator chain.

The largest bundle at 1.0T is `stackv2-edu` at 6,361 shards = **159B tokens**. At A1's measured **0.328 M tok/s/vCPU**, its tokenize time depends entirely on how many vCPU that one child gets:

vCPU for this child	tokenize	read (10 Gbit/s)	total	fits the 6.61 h floor?
8 (the wave shape in §9 Phase 3)	**16.83 h**	0.16 h	**16.99 h**	**NO — 2.6× over**
16	8.42 h	0.16 h	8.58 h	no
32	4.21 h	0.16 h	4.37 h	yes
32, at 2.5 Gbit/s instead of 10	4.21 h	1.27 h	5.48 h	yes
8, file-sharded 4 ways (32 vCPU total)	**4.21 h**	0.04 h	**4.25 h**	**yes**
8, file-sharded 8 ways (64 vCPU total)	2.10 h	0.02 h	**2.12 h**	yes

⚠️ **Corrected 2026-08-07, and this is a real defect in the earlier draft, not a presentational one.** The 4.21 h and 5.48 h figures were computed at **32 vCPU** while this same section specified **8 vCPU per child, 16 concurrent**. Those two statements are inconsistent: at 8 vCPU the bundle takes **16.83 h**, which does not merely miss the 6.61 h floor — **it is longer than the entire as-configured build**, and it would have been discovered only when the array's last child was still running eleven hours after the others finished.

⚠️ **The read column varies with BANDWIDTH only — not with `_CHARS_PER_TOKEN`.** An earlier draft labelled these rows "*as-is (9.0 chars/token)*" versus "*fixed constants*", which resurrects the constant §3.1 **withdrew**. There is no constant to fix here: the 1.27 h → 0.16 h difference is 2.5 Gbit/s versus 10 Gbit/s, and `_CHARS_PER_TOKEN` does not appear in it, because the budget is a ceiling `pack` never reaches. **Both numbers are bandwidth. Leave the constant at 9.0.**

The resolution, and it changes what task #25 means. Two things are true at once and the earlier draft conflated them:

- **Aggregate:** 1.0T ÷ (128 vCPU × 0.328 M/s) = **6.61 h**. That is the floor and it is real.
- **Per child:** one bundle's duration is *its own* tokens ÷ *its own* vCPU. A 159B bundle is **15.9%** of the corpus, so it needs ≥15.9% of the 128 vCPU — **at least 21 vCPU** — merely to finish when the aggregate does.

So the file-shard is **not** an optimization that buys 6.6 h → less. It is what makes **6.6 h reachable at all** under the stated wave shape. Either shard the big bundles or give them ≥32 vCPU each; the graph's `BUILD` = 6.6 h assumes one of the two, and §9 Phase 3 as written provides neither.

Tokenize is 96% of it once the read is fixed (0.16 h against 4.21 h at 32 vCPU), which is why per-bundle vCPU allocation — not read bandwidth — is what to get right here. `_shard_slice` is already imported at `corpus_build.py:92` and already used at `:676`, so the mechanism exists; what is missing is its application *within* a bundle rather than across bundles (see §8A.5a).

8A.5a ⚠️ `--shard/--of` slices bundles, not files — so the fix needs code

`_shard_slice(bundles_of(plan), shard, args.of)` **at** `corpus_build.py:676` **strides the list of BUNDLES.** MEASURED-IN-CODE. So today an array of 16 children distributes ~100 bundles across them — **one bundle always lands entirely inside one child**, and a 159B bundle is therefore a 16.83 h child no matter how many children the array has.

The units are worth stating plainly, because three different things are called sharding here:

unit	what it is	mechanism	status
shard	one ~25M-token `.u32le.bin` object	`SHARD_TOKENS`	exists
bundle	one (source, domain, split) stream, N shards	`--shard/--of` strides these	**exists**
file-shard	one bundle's *source files* split across children	—	**DOES NOT EXIST**

`_shard_slice`'s own docstring is about the third case — it explains striding "*FinePhrase's files*" by name — but its two call sites both pass bundle lists. The primitive is right; nothing calls it on files.

What the change requires: a bundle must be splittable into K children that each read a disjoint slice of its source files and write a disjoint ordinal range. Ordinals are the hard part, not the reading — `allocate_ordinals` walks the plan with one counter per split (`corpus.py:352–359`), so K children of one bundle must receive their ordinal ranges **from the plan**, at plan time, not negotiate them at runtime. **That**

makes it a plan-shape change, which puts it before `FREEZE` — not the ~30-line reader tweak the earlier draft implied.

And `stackv2-edu` **is not the binding case — DCLM is, by a wide margin.** §5.2a puts DCLM at **410B tokens in one bundle**, because its `domain_column` is `None` so it does not fan out. Even given an **entire** `c7i.8xlarge` **to itself — all 32 vCPU, the whole compute environment's instance type** — that one child takes **10.85 h**:

bundle	B tokens	at 8 vCPU	at 32 vCPU (a whole instance)	shards needed to reach 6.6 h
DCLM-baseline	**410.0**	**43.4 h**	**10.85 h**	**≥ 8 ways**
FineWeb-Edu	252.0	26.7 h	6.67 h	≥ 5 ways
code (`stackv2`)	108.0	11.4 h	2.86 h	≥ 2 ways
FinePhrase, one partition	36.0	3.8 h	0.95 h	1 (fits)

So this is not an optimization at any wave shape. The 128 vCPU cap is *on one instance type*, and a single Batch child cannot exceed one instance, so **no vCPU allocation makes a 410B single-child bundle fit the 6.6 h floor.** Splitting it is the only lever. That is why item 3b is on the critical path and item 3 (val bundles) is not.

⚠️ **One alternative worth considering before writing the code**, because it may be free: DCLM has a natural fan-out the plan currently discards. The parquet mirror is **27,938 files** (§4.1), and a `domain_column` — even a synthetic one derived from the file index — would make `allocate_ordinals` emit separate bundles with no new mechanism at all. That trades a schema decision (a `domain` label that is not semantically a domain, which pollutes `PATH_LABEL_KEYS`) against ~1–2 days of ordinal-range work. **Decide this before** `FREEZE` **; both options change the plan.**

8A.6 Why it is 36 h and not 10 h today: every worker default is 1

This is the systematic version of the owner's observation, and it is not scattered — it is one pattern.

Five threading facilities exist in the package and all of them work. **Every default is 1.** The build CLI exposes exactly one as a flag (`--hash-workers` , `corpus_build.py:968`); `validate.py` 's `head_workers` and `publish.py` 's `hash_workers` / `copy_workers` have **no CLI flag on this path at all**.

The cleanest example. `verify --deep` ran **1.005 TB in 3.27 h at 87.8 MB/s — one stream, on a 16 vCPU box** — finishing with 22% margin against a 4 h timeout. `verify-job.json` blames "single-threaded" code. That is not the cause: `_run_deep_rehashes` (`corpus_receipt.py:865`) supports a pool, **the fix shipped in** `0.7.5` **and was MEASURED at 7.82×**, and we are on `0.9.1` . The same run is ~25 min.

So the blocker is the job definition, not the code. `PUBLISH-SPEC.md:168` says it outright: " `edullm-reservoir-verify:1` *does* ***not*** *pass the flag — a new revision is needed before the speedup is reachable.*"

> **Therefore every wall-clock figure in this plan must name the job definition and the flags it passes, not just the stage.** A stage can be fixed, shipped, and still run at the old speed because the registered job def never passes the flag. That is how a 25-minute job stays a 3.27-hour job across a version bump.

8A.7 Error bars — what makes each of these 2× worse

projection	what would double it
tokenize 6.61 h	**linear vCPU scaling is an ASSUMPTION.** A1 was measured at 32 vCPU only; 128 vCPU on one host may contend on memory bandwidth. Never measured at the cap
read 0.26–4.0 h	**the most likely 2×, and possibly 10×.** Every read figure borrows ~85 MB/s from a *single-stream S3* measurement. Reconciling the measured 8 h reservoir build instead implies **HF CDN throughput near 8.4 MB/s** — an order of magnitude lower. Our only other datapoint is 0.8 MiB/s *out of region*
the 13-gram decon scan	**~193 billion Python-level `blake2b` calls, NEVER measured.** I attributed all CPU in the build to tokenization; this is a second CPU consumer of unknown size and plausibly explains unaccounted hours
publish	**no in-region `publish()` duration has EVER been measured**, and it is the one stage pulling ~4 TB through a client. The only datapoints are 0.8 MiB/s from a laptop off-region and a 125 GB Batch publish that **timed out at 3600 s** single-threaded with 31 of 32 vCPU idle
Gate A 0.36 h	threading gains are capped by `max_pool_connections` (default **10**); at 16 workers it self-throttles unless the pool is raised too
`verify --deep` 1.66 h	7.82× was measured at 1.005 TB; at 4.0 TB the read may saturate the NIC before 8 streams. **Where that 7.82× was measured is not recorded**
all of it	**the 128 vCPU cap is a queue property, not a physical one.** If the queue is shared, effective concurrency drops and everything scales inversely

Never measured at the target scale: Gate A beyond 10,049 objects, `verify --deep` beyond 1.005 TB, tokenize beyond 32 vCPU, `publish()` in-region at all, the decontamination scan's CPU cost at all, and `_reader_for` against live HuggingFace from inside a Batch container at all.

One job settles four of those bands at once — the mandatory single-bundle smoke test (§9 Phase 2) combined with Phase 0b's bandwidth measurement. **Run it before trusting any figure in this section.**

A counter to my own confidence here. An earlier session's per-bundle ETAs during the real run were **all retracted** by their own author: "*every per-bundle ETA I gave was wrong, in both directions.*" Two of my own numbers in this document were also wrong until a later audit caught them (§3.1's read volume, and the Gate A call count in §8.1). **Treat the "fixed" column as a target to verify, not a promise** — the *ordering* of the fixes is far better evidenced than their absolute durations.

---

9. The job plan

> → **For the parallel execution order, see** `docs/BUILD-DEPENDENCY-GRAPH.md` — a 33-node DAG with the critical path computed (**13.31 h**, against ~36 h as configured and 17.31 h parallelized naively), plus an orchestrator brief that can be handed to an agent verbatim. Its central finding is that the binding constraint on parallelism is **file contention, not logic**: five code items edit `corpus_build.py`, so an agent must own a *function*, never a file.

Nothing here auto-publishes. Every AWS job goes through the platform submission form and needs a human release.

Phase 0 — code and measurement, no AWS, ~2 days

All of it is pure code or a read-only query, and **all of it must land before the plan is frozen**, because several items change the plan.

Numbering. The `#NN` column is the session task id, `graph` is the node in `BUILD-DEPENDENCY-GRAPH.md` §2. Three schemes were in use with no crosswalk; this table is the crosswalk, and it is authoritative. **Items marked CHANGES THE PLAN must land before** `FREEZE`, not merely before the first job.

#	item	task	graph	why it is here	effort
1	**Wire the FinePhrase id partition** into `_reader_for`	#21	**C1**	**CHANGES THE PLAN.** Cannot be retrofitted after tokenization	~5 lines + the budget correction below
2	~~Correct `_CHARS_PER_TOKEN` ~~	—	—	**WITHDRAWN — see §3.1.** The budget is a ceiling never reached, so this saves zero bytes and starves bundles if set too low	—
3	**File-shard the VAL bundles** via `_shard_slice`	#25	**C3**	43% of bytes moved serves 0.39% of tokens. **DEFERRABLE** — pure read-volume saving	~30 lines
3b	**File-shard the BIG bundles** (§8A.5a)	**#28**	**none — no node**	**NOT deferrable and NOT the same fix as 3.** Without it the 159B `stackv2-edu` child is **16.83 h** at 8 vCPU and `BUILD` cannot hit 6.6 h. Needs plan-time ordinal ranges, so it **CHANGES THE PLAN**	**~1–2 days**
4	**Replace the dedup set with a flat** `np.uint64` **pre-pass**	#22	**A2a + A2b**	The current design OOMs at 1T (§5.2a: DCLM needs 27.92 GB)	~1 day
5	**Pin** `tokenizers` in `pyproject.toml`	#23	**B1**	Production currently resolves whatever PyPI serves	1 line
6	**Thread the profile checks + raise** `max_pool_connections`	#10	**B3**	Gate A does not fit otherwise	**~100 lines** (see below)
7	**Record** `FilterStats` **in the receipt**	—	none	Without it no dedup claim is auditable	~10 lines
8	**Fix the boundary-marker prefix guard**, or comment why the table must stay at one entry	—	**B2**	A future addition to it is silently a no-op today	~5 lines
9	**Drop** `data_provenance_initiative`	#16	**B4**	Ships GSM8K in CoT format; costs 0.51% of tokens	registry edit
10	**Query the live validator timeout**	#11	none	`edullm-validator:12`'s timeout is recorded nowhere	one read-only call
11	**Decide** `SHARD_TOKENS`	**#9**	**B6**	**CHANGES THE PLAN** (§8.2). The code says 25,001,984; confirm and stop carrying two values	~1 h
12	**Rebuild the decontamination index from raw fields**	**#24**	**B5**	Blocker 4 (§6.2). **Gates** `FREEZE` **with only ~0.9 h of slack** — start it early	~4 h

Two items the earlier draft omitted and the graph treats as gating: #11/B6 and #12/B5. B5 in particular has the least slack of anything off the critical path.

⚠️ **Item 6's effort was stated as ~20 lines here and ~100 lines in `pipeline-scale-audit.md`.** The audit is right and this table was wrong: threading the profile checks means adding a thread pool to `pretrain_tokens_v1.py`, which has **zero** threading today, *and* raising `max_pool_connections` in `s3.py`, *and* keeping the per-key size cache correct under concurrency. **Budget ~100 lines.**

⚠️ **Item 1 has a trap worth naming:** applying the partition without dividing the reader's character budget by the keep fraction means every bundle finishes and *then* fails `verify` on unfilled shard refs. The two changes ship together or not at all.

⚠️ **Do not confuse item 1's budget change with the withdrawn item 2.** They point in *opposite* directions. Item 2 would have made `_CHARS_PER_TOKEN` **smaller** (9.0 → ~4.3) and is withdrawn because the budget is a ceiling. Item 1 makes the budget **larger** — dividing by the keep fraction — because the partition discards ~72% of what it reads and the reader must still fill its shards. Applying item 2 while doing item 1 starves every FinePhrase bundle.

Phase 0 wall-clock: ~3–4 days of engineering, zero AWS. Items 5, 8, 9 are one-liners; items 1, 7, 11 are half-day changes; items 4, 6, 12 are ~1 day each; **item 3b is the new 1–2 days** and it is on the critical path. Item 10 is a single read-only call. Item 3 is deferrable.

Phase 0b — three measurements that gate specific sources — ~2 h of compute, plus a human

measurement	what it decides	wall-clock
In-region S3 read bandwidth	Whether tokenization is on the critical path; decides the gigatoken question and every read estimate in §8A	**~10 min**
Nemotron-CC-Math's real dolma2 token count	Its 133B is CARD with no tokenizer named, and it is **gated** — a human must accept the license before its text column can even be named	~5 min once ungated; **human-blocked**
Dolma 3 adult-content prevalence	Blocking for that source. Sample at **random offsets** — a prior attempt could not separate the signal from HuggingFace preview ordering	**~1 h**
mean doc length for 5 stage-2 sources	The dolma3 QA source (GPT-4o-mini-rewritten multiple choice) is the one plausibly near the 20-token EOS floor	**~1 h**

Run the bandwidth measurement first. It is ten minutes and it calibrates every other number in §8A — and per §8A.1, an assumed bandwidth is exactly the mistake the calibration file made.

Phase 1 — freeze the plan, then stage — 0.5–3.0 h

1. **Freeze the mix.** Every source, both stages, final shares. **This is the real gate, and its duration is a decision, not a computation.**
2. **Generate the complete plan.** Pure function, no network — **seconds.** Record the `plan_id`.
3. **Stage ~4.21 TB to `s3://edullm-landing/_src/`** — **0.5 h** with parallel copy children, up to 3.0 h serially. ~$194 for a two-month window; inbound transfer is free.

Phase 2 — ⚠️ a mandatory single-bundle smoke test — ~20 min

`_reader_for` has never run against live HuggingFace from inside a Batch container, and the code says so (`corpus_build.py:901-904`). Run one bundle against the smallest source — `ubuntu-irc` at 1.75B tokens / 71 shards, the smallest real bundle in the reservoir — before committing an array job.

Twenty minutes to de-risk a 6.6-hour array. The cheapest item in this plan.

Phase 3 — the build, in waves — 6.6–11.2 h

~100 bundles across 4–6 array waves at **8 vCPU each = 16 concurrent** under the 128 vCPU cap. Per-bundle resume already works: `bundle_is_done` re-HEADs and compares sizes, and re-running a lost bundle is byte-identical (verified — nine bundles reproduced identical digests).

6.6 h is the CPU floor (§8A.3). The 11.2 h upper figure is what an unsliced run costs, because the 159B-token `stackv2-edu` bundle alone is a 4.37–5.48 h single child.

Job def must pass: enough vCPU per child to matter, and the wave shape. Nothing else here is flag-dependent.

Phase 4 — publish two datasets — 0.3–2.0 h publish, then validate

`pretrain/final-stage1-900b` and `pretrain/final-stage2-100b`. Per stage: publish, Gate A, `verify --deep`, promote.

step	stage 1 (~36,000 obj)	stage 2 (~4,000 obj)	job def must pass
publish (hash + copy)	0.3 h threaded / ~2 h at 1	0.03 h / 0.2 h	`--hash-workers`, `--copy-workers`
Gate A validate	**0.32 h** threaded / **5.08 h** ❌ **serial**	0.04 h / 0.56 h	needs the §8.3 code fix first
`verify --deep`	**1.49 h** at 8 workers / **11.7 h** ❌	0.17 h / 1.3 h	`--hash-workers 8`
promote	~1 min	~7 s	already threaded

Both ❌ figures exceed their job timeouts, so they are failures rather than slow runs.

⚠️ **Writing a `manifest.json` fires EventBridge and promotes automatically.** To stage without promoting, cancel the validator job or disable the rule first. (On the reservoir the rule was left **disabled**, so nothing auto-promoted and the validator was submitted by hand — confirm which state the rule is in before Phase 4.)

Total: ~10 h fixed, ~36 h as-configured

The 3.7× gap is entirely flags and two constants — no new architecture. Add **~2 days of Phase 0 code** and the calendar cost is dominated by approval latency, not compute.

---

10. What is still unverified

Listed because the distinction between measured and inherited is the only thing that makes the rest trustworthy.

Blocking a specific source:

- ✅ **RESOLVED 2026-08-07 — Nemotron-CC-Math's token count is now MEASURED**, by a teammate with gate access. **134.0B under dolma2**: 472,213,218,716 uncompressed text bytes from exact parquet footers × 0.283686 tok/byte sampled over 1,920 random-offset documents at seed 42. Per config: `3` ≈ **83.6B**, `4plus` ≈ **50.4B.** The card's 133B was close. Artifact: `_nemotron_cc_math_dolma2_measure.json`. The implied **3.53 bytes/token** is denser than English prose's ~4.31 on this tokenizer, which is the expected direction for LaTeX-heavy text.

Still open on this source, and none of it blocks the count:

- **The `text` and id column NAMES are still unconfirmed in writing.** The count was produced by reading the text column, so somebody knows it — **record the exact `text_column` and id column before the registry row is written.** §4.2 is the cautionary case: FinePhrase has *two* plausible `text` columns and the flat scan picks the wrong one silently.
- **`3plus` is still not a loadable config** — it is the union of `3` and `4plus`, so the registry needs **two rows**, not one. The measurement's per-config split confirms this shape.
- ⚠️ **`4plus_MIND` is a third config and must NOT be added to the math pool.** It is a *rewrite* of `4plus` (the card lists all three), so including both double-counts the same documents — the same trap §4.2/§4.3 describes for FinePhrase, and the same reason §6 of the report treats math as one artifact. 134.0B is `3` + `4plus` only, which is correct. **If `4plus_MIND` is being uploaded, label it so it cannot be swept into the pool by a prefix match.**
- **Dolma 3's adult-content prevalence** is unmeasured and blocking.
- **The dolma3 midtraining mix is `allenai/dolma3_dolmino_mix-100B-1125`** and ships **`.jsonl.zst` text**, not pre-tokenized shards — so re-tokenization is required, filtering is possible, and **zstd is unreadable by `corpus_read` today**. Its 323 directory names *are* category labels, so QA is selectable by prefix with no classifier. Note `allenai/dolma3-dolmino-mix-1025` does not resolve.

Structural unknowns:

- **Three sources have no usable document id** — both full-DCLM repos and Cosmopedia. Every `sha256(id) % N` partition and anti-join in this plan depends on one. Decide the surrogate before ingest; a hash-of-text id is not stable across a re-download.
- **252B of FineWeb-Edu breaks the FinePhrase anti-join.** That volume must come from `sample-350BT`, which is FinePhrase's exact parent, so ~72% of the rephrase source lands in the corpus as real text too.
- **FinePhrase ships `finish_reason` and the plan ignores it.** `max_tokens=2048` with a measured p99 rewrite length of 1,847–1,948 tokens, so ~0.5–1.5% of faq/tutorial rewrites end mid-sentence — concentrated in the highest-token-mass documents. Nemotron-CC-Math, Cosmopedia and Nemotron-Specialized have the same exposure with **no such field at all**.
- **Per-source normalization is inconsistent and invisible to every validator.** `finepdfs-edu` already ran FTFY, so it is NFC-normalized, ligature-split and straight-quoted; no other source is. And **peS2o is a PDF-extraction source in disguise** — Grobid over PDF, with OCR errors "filtered not fixed" and no FTFY.
- **Cosmopedia's leading space, MEASURED at 303/303 documents across all 8 configs.** A Mixtral SentencePiece `▁` detokenization artifact. Under byte-level BPE the space-prefixed and bare forms of a word are **different token ids** (verified against the real dolma2 vocab: `The` = 791, `ĠThe` = 578), so every document's first token is its mid-sentence variant, immediately after our appended EOS. One `lstrip()` fixes it — **but it must not be applied globally, because leading whitespace is semantic in code.**
- **Never unescape `\n`.** Real Nemotron bytes contain `\neq` and `\nabla`; the obvious heuristic would destroy exactly the math the source is being bought for.
- **Gate A's cost at ~40,000 objects** is extrapolated from a measured 85 min at 10,049. Never tested.
- **MegaMath-Web's post-filter pool size** is unmeasured.
- **`c7i` pricing** is quoted from memory, not a live price API.

The largest residual risk, unchanged: every mixture study behind the shares in `FINAL-DATASET-REPORT.md` was run on a **dense** model. **No mixture ablation on a sparse MoE exists at any scale.** Transfer to our architecture is unverified, and that unknown is larger than any measurement error in this document.

A methodological caution that applies to several numbers above: the Cosmopedia and Nemotron measurements come from HuggingFace `/first-rows`, which is a head read. A prior wave found head sampling on content-clustered parquet wrong by **10×** once already. What makes the Cosmopedia result convincing is the **100% rate across 8 independent configs**, not the sample size.

One free improvement worth taking: a per-shard **encoding receipt** — a handful of O(1) predicates bolted onto the tokenize path (does any document start with whitespace, does any contain `<|endoftext|>`, is any text non-NFC). It converts eight of the UNVERIFIED rows above into MEASURED ones as a side effect of a build that was going to read the bytes anyway.